RESEARCH PROPOSAL

Image Recognition Models On A Resource-Constraint Federated Learning Network of Heterogeneous Devices Using Real World Data

Author:

Olena Pleshan

sbs24043

11 November 2024

Table of Content

List of Figures..... 3
1. INTRODUCTION..... 4
1.1. Research Motivation..... 4
1.2. Research Objectives..... 5
2. LITERATURE REVIEW..... 5
2.1. Introduction..... 5
2.2. Model Optimization Techniques..... 5
2.3. Model Optimisation for Microcontrollers..... 6
2.4. Federated Learning..... 7
2.6. Image Recognition..... 8
2.7. Research Opportunities..... 8
3. RESEARCH STRUCTURE..... 9
3.1. Primary and Secondary Research..... 9
3.2. Sampling Strategy..... 9
3.3. Methods of Research..... 10
3.3.2. Data Collecting and Preprocessing..... 10
3.3.3. Experiments..... 11
3.3.4. Evaluation and Analysis..... 11
3.3.5. Pipeline Documentation..... 11
4. ETHICS AND LEGAL REGULATORY CONSIDERATIONS..... 12
4.1. Legal/ Regulatory Considerations..... 12
4.2. Ethics and Data Protection..... 12
REFERENCES..... 13
APPENDIX..... 17
Terminology..... 17
Hardware Technical Specification..... 17
Experimental Flowchart..... 18

List of Figures

Table 1. Federated Learning Frameworks under Apache-2.0 license

Table 2. Hardware Technical Specifications

Figure 1. Experiments Flowchat

1. INTRODUCTION

This research explores image recognition on edge devices, an area of growing importance in sectors like agriculture, manufacturing, mining, and rescue operations, where autonomous systems rely on image recognition to navigate and make decisions in real time. In applications such as robotics, autonomous vehicles, and drones, local image recognition enhances decision-making speed, cost efficiency, accuracy, and data privacy while reducing reliance on cloud processing. Edge analytics also enables real-time responses essential for critical applications like autonomous driving, security, and medical monitoring, especially in environments with limited connectivity.

As machine learning evolves toward distributed models, edge learning - an approach rooted in distributed machine learning (DML) - has emerged. Edge learning allows multiple devices to train models collaboratively without a centralized server, addressing privacy, security, and the challenge of heterogeneous data from varied devices. Federated Learning (FL) further refines this by enabling distributed learning across mobile devices where data remains decentralized, promoting privacy and security.

This research focuses on advancing image recognition model training and optimization for edge devices. Specifically, it investigates the use of Transfer Learning (TL) and FL across a network of heterogeneous devices, with model optimization techniques like quantization and pruning for deployment on microcontrollers. While substantial work exists on real-time image recognition, few studies have implemented an end-to-end proof of concept using TL and FL on diverse edge devices with non-reference data input and optimized models for microcontroller deployment, a gap this research aims to address.

1.1. Research Motivation

Machine learning models are traditionally assumed to be trained in cloud on clusters of machines, on specialized or highly optimized hardware. Edge analytics is at its infancy (Zhang, Qu et al, 2021), and the majority of research papers have appeared in the past 5 years, and further research is required to address real world performance of different models where transfer learning, federated learning and model optimizations have been used.

Federated learning helps maintain data privacy by keeping the data on the device rather than transmitting it to central servers, which is useful in terms of privacy and critical in the environment where internet connection and long range data transmission is impossible (agriculture, mining, etc) (Abreha et al, 2022). It also allows continuous model improvement across distributed devices without directly accessing raw data, which is valuable for adapting models to diverse, real-world environments - however, current studies have a gap whereby, a lot of training is done on reference datasets and real world performance is not investigated enough (Beltrán, 2024).

Therefore, this research is motivated to address the following issues:

1. Training, fine-tuning, and evaluating models using real-world image data, including data generated in real-time.
2. Implementing continuous training on edge devices through Federated Learning across heterogeneous devices.
3. Deploying optimized models from these processes onto microcontrollers.
4. Validating and benchmarking the effectiveness of Docker containers for training Federated Learning models with real-world datasets.

1.2. Research Objectives

One of the major problems for training ML models on edge devices is device heterogeneity (Iqbal, 2024). Additionally, most research is being done on reference devices such as Raspberry Pi and reference datasets such as CIFAR-10, with some attempted end-to-end implementations for particular use cases (Hayajneh et al, 2023). Therefore, we set out the following research objectives:

1. Develop an end-to-end proof of concept system capable of training an image recognition model using Federated Learning and Transfer Learning on a peer-to-peer network of resource-constrained devices, optimizing the output model and deploying it to a microcontroller.
2. Develop an aerial recognition neural network model using the novel benchmarking RF100 dataset, custom made open-source datasets and datasets coming from real time data collection, aiming to achieve an accuracy of above 90%.
3. Establish a thoroughly documented pipeline detailing the assembly and deployment of the end-to-end system, including both software and hardware components.

2. LITERATURE REVIEW

2.1. Introduction

The shift toward edge computing seeks to enable real-time image recognition and data processing on compact, resource-limited devices like microcontrollers, Raspberry Pi / Radxa Rock Pi units, smartphones, etc. Current research explores a wide range of techniques such as model quantization, pruning, and hardware-aware optimizations tailored to the limitations of edge devices. Federated learning (FL) has also emerged as a key approach to secure distributed model training, allowing for data privacy while reducing reliance on centralized servers and network connectivity. This literature review delves into these techniques, highlighting current advancements, and identifying the gaps that still limit efficient model deployment and training on edge platforms .

2.2. Model Optimization Techniques

Model quantization involves reducing the precision of weights and activations, and can be performed during the conversion of a retrained model (post-training quantization), or integrate quantization constraints during the training process (quantization-aware training) (Iqbal, 2024). Depending on the hardware complexity and off-chip memory available on the target device, quantization can be either aggressive through binarized quantization which quantizes each weight to a binary value, or a multi-bit quantization which can allow for 8, 16 or 31 bit point flotations (Liu et al, 2021). Hybrid precision / non-uniform quantization, which offers a varying bit precision for different model components, is a recent development in the field, however, accuracy remains a challenging factor for CV applications (Iqbal., 2024).

Network pruning is another technique for model optimization, which involves the elimination of neurons, weights (structured pruning) and channels (channel pruning) to significantly reduce computation and space complexity (Liu, 2021).

Knowledge distillation is widely used to compress and accelerate deep neural networks in AI fields like visual recognition, speech recognition, NLP, and recommendation systems. This vast

area employs distillation of different types of knowledge (model outputs, weights, etc), different architectures and schemes, and results of applying these methods have a significant impact on the model accuracy, as described in Gou et al, 2021.

Hardware-aware optimization is a technique which aims to efficiently utilize the specific architecture of target edge devices, which is what most current libraries fall short due to them being tightly coupled to their software implementation. The problem has been tackled using the notion of a deep learning compiler enhanced with a new graph-level dialect called Quantized Neural Network (Jain, 2020). Another promising development in this space, for example, is Glow compiler, which offers an approach to compilation that enables the generation of highly optimized code for multiple targets. Glow lowers the traditional neural network dataflow graph into a two-phase strongly-typed intermediate representation, with the high-level intermediate representation doing domain-specific optimizations, and the lower-level instruction-based address-only intermediate representation performing memory-related optimizations (Rotem et al., 2019).

2.3. Model Optimisation for Microcontrollers

Multiple research has been done on further optimizing models for making inferences on microcontrollers (Hayajneh et al, 2023), which is called TinyML (Róbert Dávid et al., 2020). The latter is the whitepaper for TensorflowLite Micro (renamed to LiteRT) which addresses unique challenges of embedded systems which are hardware heterogeneity and severe resource constraints. Binarized neural networks are a type of optimized networks specifically deployable to microcontrollers, and in addition to Tenforflow Lite, libraries like qKeras and Larq have shown good retention of accuracy in the binarized models (Pau et al, 2021). According to Pau, binary neural networks give the greatest advantage when it comes to memory resources, as they use 1-bit to store activations, weights and biases. One of the limitations of the pretraining models for and deploying to an MCU is, however, architecture of some neural networks which require multiple graphs for their internal memory cells (Hayajneh et al, 2023). Hence, a network architecture needs to be carefully chosen even while training a model on a resource intensive device, for future deployment using an optimized framework.

Multiple papers discuss end-to-end implementations of machine learning models for edge. The implementations vary in there use cases (surveillance, agriculture, etc), input data used (camera images, moisture / temperature sensors, etc), and target devices which vary from Raspberry Pi's of different generations and models to microcontrollers.

Some experiments have implemented face recognition and other CV algorithms using Tensorflow and Tensorflow Lite on Raspberry PI (Andrea, Ikhsan and Sudirman, 2022), with the model itself being trained on the SBCs directly and converted into Tensorflow Lite for inferencing. Other research proposes an architecture in which the models (in this case CV CNN) is pretrain during Google Colab and a Desktop environment, and Raspberry Pi only receives a TF lite converted model (Pandey & Abhijit, 2023).

Another edge architecture is to have microcontrollers or micro devices communicate with a worker device (mobile sink / data mule) which does round trips to a centralized of more powerful platform for data delivery and ML inferencing, or have a movable device act as an edge device and makes ML inferences (Rezende, Silva and Souza, 2020). This can be done using unmanned aerial vehicles due to their mobility (Hayajneh et al, 2023). Drone (or inter-connected chain of devices) could act as enablers for a self-organized network and could

perform data collection, real-time training, distributing the learned model, etc. An improvement on the architecture proposed by Hayajneh et al, could be the use of data mules proposed by Rezende et al to deliver the data to the centralized server for training instead of communicating with the server directly.

2.4. Federated Learning

In response to communication costs, data privacy and legalization, federated learning was developed as an algorithm to address those challenges, and is deployed by Google, and other companies (Bonawitz, 2019). Federated learning aims to train ML models in a distributed manner, where an edge or cloud server coordinates multiple devices to gather the trained model parameters, update them and create a better model before sending it back to the devices, with the process happening over multiple iterations (Abreha et al, 2022). FedAvg is the commonly used aggregation algorithm, while new architectures are being proposed such as EdgeFed, which introduces an edge server layer which coordinates split training for edge devices (Ye, 2020). New algorithms for federated learning on edge devices are being explored, since heavier computation tasks such as image recognition, since the simplistic algorithms are only able to evaluate model converges on small CNNs, produced by the client devices and training larger models is impossible due to resource constraints (He, 2020). Hence, research is seeing a myriad of new algorithms such as FedProx, SplitFed, FedGKT, FedDCT (He, 2020; Nguyen, 2023). Table X provides an overview of Federated Learning frameworks which have been considered for the practical experiments of this study.

Framework	Maintaner, # Contributors	Distinguishing Features
PySyft	OpenMined, 428	Innovative Remote Data Science concept
Tensorflow Federated	Google, 111	Used at Google for customer needs. Customizable, programmable Federated Core API, and a higher level Federated Learning API
Flower	Flower, 137	Flower Baselines are community contributions which reproduce experiments described in scientific papers
Nvidia Flare	NVIDIA, 45	Variety of Federated Learning algorithms and support for multiple ML frameworks such as TF, Pytorch, Scikit-Learn
FATE	WeBank 87	Multiple related projects to tackle specific scenarios such as building LLMs, or running on cloud native (Kubernetes) services. Positions itself as industry-grade
OpenFL	Linux Foundation & Intel, 94	Uses mTLS, supports data compression, customizable, and uses Docker containers

Table 1. Federated Learning Frameworks under Apache-2.0 license

The techniques mentioned above are known as Centralized Federated Learning (CFL), but they have limitations, such as communication bottlenecks and a centralized server being a single point of failure. As a result, Decentralized Federated Learning (DFL) is emerging as a solution. Beltrán et al. (2024) developed FedStellar, a highly configurable DFL solution that supports different network topologies, aggregation algorithms, and device heterogeneity through Docker containers. Knoche (2018) found that Docker had minimal impact on processing and throughput on Raspberry Pi. The ProxyFL approach introduces another peer-to-peer architecture where only proxy models are exchanged, enhancing privacy (Kalra, 2023). Recent studies have shown that both Centralized and Decentralized FL models can run effectively on edge devices like Raspberry Pi (Ridolfi, 2023; Karras, 2022). Lastly, combining Transfer Learning with Federated Learning has proven effective for edge devices. Zhang et al. (2021) developed a framework for federated transfer learning in IIoT, where transfer learning addresses data scarcity and federated learning creates relevant models on edge devices.

2.6. Image Recognition

Image recognition is a widely studied area of data science. One of the most challenging aspects of object detection in real-world images is that objects rarely appear in isolation. This makes detection difficult, as distinguishing individual objects within an image is complex (Poda et al, 2018). According to Brahmbhatt, 2013, image segmentation from OpenCV could be used to ensure we are plotting out the regions of interest and extracting objects which we would like to examine. A lot of novel research is performed for image segmentation, in particular, neural networks are proposed as one of the promising tools (Yu, 2023). Practical implementations for image segmentation and image recognition can be found available in open source (Abdulla, 2017; Carion et al, 2020). Literature also points at successful implementations of computer vision tasks on microcontrollers such as Arduino (Ozkaya, 2015), which is limited and challenging but appears to be achievable.

2.7. Research Opportunities

All of the described approaches have certain gaps and require further research. Energy efficiency and computational complexity remain a challenge, which requires solutions at both software and hardware level. Model maintenance and management has its complications on edge devices due to the necessity of continuous model training (Iqbal, 2024). Secure communication between edge devices is an engineering gap that is often cited, and research is being done around securing those communications - this is specifically applicable to distributed ML use cases (Zhang et al, 2021).

An important existing opportunity with edge devices is their heterogeneity, i.e. diversity in hardware and software systems supported. While some research proposes and successfully implements the use of containerization and virtualisation for model deployment (Beltrán et al, 2024), this is a novel field and further research is required to expand these evaluations to various data types and network topologies.

When it comes to Federated learning, the data on edge devices that participate in FL training are not independently and identically distributed (IID), resulting in challenges related to heterogeneity data (Lu, 2024), which add another complication.

Another research gap is the reliance on reference datasets like MNIST, CIFAR-10, and

CINIC-10 for much of the development work (He, 2020; Beltrán et al., 2024; Gou, 2021; Ridolfi, 2023), with limited studies using real-world datasets. This reliance, in our view, poses a significant barrier to identifying further gaps and inconsistencies when these techniques are applied in real-world settings. The creation of RF100 (Ciaglia, 2022) and the over 500.000 labeled datasets from the real world, allow for further research to try and close that gap.

3. RESEARCH STRUCTURE

3.1. Primary and Secondary Research

For the purposes of this research, we propose to utilize datasets from an open-source Roboflow computer vision dataset repository, pioneered by Ciaglia et al in 2022. In particular, for creating the base model (*Model T*), we will be using the Arial category from *RF100 benchmark dataset* which presents 24 classes (roboflow, 2022), and is a verified, Intel-backed real-world dataset.

Model T will serve as primary data for another model (*Model F*) which will be refined / fine-tuned with the *dDrone dataset* (Ddrone, 2023). The dDrone dataset is unvalidated, therefore validity of the dataset will need to be assessed for accuracy, consistency and quality by:

1. Performing random sampling of a subset of images.
2. Use Roboflow manual annotation services and have human reviewer labels the sample. We will employ multiple annotators to label the same subset of images, and then calculate inter-annotator agreement (e.g., using Cohen's kappa) to measure labeling consistency.
3. Check the labels produced in step 2 against the original labels in the dataset.

According to Northcutt et al, 2021, research estimates that label errors comprise at least 6% of the ImageNet validation set, while CIFAR-10 test data is estimated to contain about 5% mislabelled examples. The research, in particular, uses human evaluation to arrive at these numbers. Therefore, following the evaluation steps described above, if the chosen dataset is found to have lower then 95% labeling accuracy, we reserve the right to select another dataset from Roboflow or re-label / adapt the existing dataset (which is within the permissions granted by the CC BY 4.0 license).

We will also determine whether the *dDrone dataset* is balanced, and balance as required.

Performance of *Model F* will be tested on images (data) collected by Raspberry Pi v2.1 8 MP 1080p Camera Module in the vicinity of the Dublin International Airport, Dublin Weston Airport, and bird watching sites in Dublin.

Model F will serve as primary data for *Models O1..n* which will be highly optimized (quantised, pruned, etc) models deployed into Arduino Sense. Performance of *Models O1..n* will be evaluated on images (data) collected by Arducam Mini Module Camera Shield with OV2640 2 Megapixels Lens in the vicinity of the Dublin International Airport and Dublin Weston Airport, and bird watching sites in Dublin.

3.2. Sampling Strategy

Since one of the research objectives here is to enable aerial image recognition detection, the sample population of interest are aerial objects such as birds, drones, planes, helicopters. Other

potential aerial objects such as missiles and wartime drones are outside of the scope of this research.

Convenience sampling is a non-probability sampling technique which selects participants from the target population based on ease of access (Golzar, 2022). Benchmarking computer vision datasets fall under this category, and have been used for experimentation in the papers described in the literature review above. ImageNet, CIFAR-10 and CIFAR-100, MNIST, COCO (Common Objects in Context), Open Images, Cityscapes and other datasets are often used for experimentation and uniform benchmarking across the research papers reviewed for this research. However, these datasets consist largely of images found on the web and do not represent many real life domains modeled in practice (Ciaglia et al, 2022).

Random Sampling is a method of selecting a subset of individuals, items, or data points from a larger population so each member has an equal chance of being included in the sample (Freund, 2003). This technique will be used for *dDrone dataset* validation, and for collecting data for validating *Models F* and *O1..n*. We will be performing *Stratified Random Sampling*, since we have 4 distinct subgroups (airplanes, helicopters, birds, drones) and will draw an equal number of samples from each group.

Undersampling or *oversampling* (Shelke et al, 2017) may need to be performed on the *dDrone dataset* in order to ensure dataset balance.

3.3. Methods of Research

In order to achieve the objectives, set out for this research, the following research methodology will be used.

3.3.1. System Design and Architecture Setup

Firstly, we will outline the hardware and software specifications of the devices in the peer-to-peer network, focusing on limitations in memory, processing power, and connectivity (see Appendix Table 2. Hardware Technical Specifications).

When it comes to determining the architecture for FL across devices, leveraging TL from a pre-trained model, an in-depth evaluation of selected frameworks will need to be done by using literature review and corresponding tables (see *Table 1. Federated Learning Frameworks under Apache-2.0 license*)

Evaluating protocols like MQTT or over-the-air model updates is outside the scope of this research, and a simple Wi-Fi or Bluetooth connection will be used for model exchange in the FL network.

3.3.2. Data Collecting and Preprocessing

For data collection, we will use RF100, custom open-source datasets, and real-time data collected from aerial sources. The methodology will focus on diversity of the data sources to address some of the key research challenges which limit the input data to curated data only.

We will perform data preprocessing and augmentation, in particular to standardize and preprocess datasets to align with model input requirements (e.g., image resizing, normalization, data augmentation) on the dDrones dataset. This will help reduce overfitting and improve model generalization.

Finally, in order to train the models using Federated Learning, we will be splitting the data into segments that reflect realistic data distributions across devices, ensuring data is non-IID (non-independent and identically distributed (Lu, 2024) to simulate real-world scenarios in FL.

3.3.3. Experiments

Experiments Flowchart (see *Appendix Figure 1. Experiments Flowchart*) demonstrates the full system setup and experiment flow.

To provide more details, here is what those experiments will entail:

1. Base Model (*Model T*) Training for Transfer Learning: We will pretrain a generic model adapted to the aerial recognition task, based on RF100 dataset, modifying layers and parameters as necessary to handle domain-specific images.
2. Federated Learning Protocol Implementation: We will implement a Centralized or Decentralised FL protocol (will be attempted on Docker and / or on the system itself) that allows a network of RASPI and ROCKPI devices to update the model locally and then share weights without sharing raw data. The devices will be using prepared and split data from dDrones dataset, and the Base model from step 1 (*Model T*). Training rounds will be logged and streamed to a centralized device for measurement and record of communication intervals to ensure model convergence, and capture performance metrics. After initial training, fine-tune hyperparameters (e.g., learning rate, batch size) to improve model performance, ensuring the target accuracy (≥ 90%) is met. This will produce a *Model F*.
3. Validation On Real-World Data: We will perform *Model F* assessment (accuracy, average precision) on real time data gathered from Pi v2.1 8 MP 1080p Camera Modules attached to each device. Step 2 may be repeated based on the results of step 3, to fine tune the models or go back to data preprocessing.
4. Model Optimization for Microcontrollers: We will perform model compression techniques such as pruning, quantization, or knowledge distillation to reduce the size of *Model F* and make it compatible with microcontrollers (output will be *Models* $O_{1...n}$).
5. Deployment on Microcontroller (Arduino): *Models* $O_{1...n}$ from step 4 will be deployed and we will *attempt* to run them.
6. Validation On Real-World Data: If the models are able to run on the Arduino MCU, data from the real world will be collected with an OV2640 Arducam Mini for model evaluation to select the most effective technique to balance model accuracy and efficiency.

3.3.4. Evaluation and Analysis

We will assess the final model's performance on accuracy, latency, and resource efficiency. Compare results to the initial objectives to ensure all performance benchmarks are achieved. Analysis will be made with regards to the scalability of the proposed system and any limitations or areas for improvement will be identified.

3.3.5. Pipeline Documentation

Finally, in order to achieve objective 3 of this research, we will use the following methodology:

1. Document the entire pipeline, including setup instructions, configurations, and troubleshooting tips for replicating the system.

2. Provide well-organized code, with comments and modular functions for each step (e.g., data preprocessing, model training, optimization).
3. Detail the hardware assembly and software deployment steps, creating a guide for setting up and launching the end-to-end system on similar devices.

4. ETHICS AND LEGAL REGULATORY CONSIDERATIONS

4.1. Legal/ Regulatory Considerations

According to Roboflow's guidelines (2024), users may not upload, distribute, or process restricted content, such as data subject to export controls, or re-host downloaded datasets without permission from the dataset author. To comply, appropriate steps will be taken for the RF100 and dDrones datasets. The dDrones and some Roboflow datasets are under a Creative Commons Attribution 4.0 International license, allowing free sharing and adaptation with proper credit and no endorsement implications, although some rights may still apply (Creative Commons, 2016).

Docker has very complicated licensing, however Docker containers can be used for educational / research purposes as part of Docker Personal subscription (Docker Documentation, 2024; Docker FAQ). Docker has been used in research without references to licensing complications (Openja et al., 2022; Beltrán et al., 2024).

Raspberry Pi hardware is available for personal, educational, and commercial use without additional licensing. Most of its source code is under BSD 3-Clause, though some may have other licenses (Raspberry Pi, n.d.). Radxa Rock Pi has no specified licensing terms, and Arduino hardware and software can be freely used personally or in organizations, with restrictions only on product distribution (Arduino Help Center, 2024).

Finally, we have looked into AI, autonomous drones and existing regulations in this area (Hartmann et al., 2022). In this research, however, images of drones and planes will be analyzed, but no hardware or software will be installed on drones or unmanned vehicles. Additionally, any models developed will not be deployed for automated actions or decision-making on devices with flying or motion capabilities.

4.2. Ethics and Data Protection

During the sampling process, we will need to ensure that no persons or private property are being captured on camera. If they are, the samples need to be discarded immediately.

The model will *not* be contributed back to Roboflow and will only be available on request, to ensure it is not used for unintended purposes, such as on drones / unmanned flying vehicles by any third party.

REFERENCES

Abdulla, W. (2017) Mask R-CNN for object detection and instance segmentation on Keras and TensorFlow, GitHub repository [Preprint]. Github. Available at: https://github.com/matterport/Mask_RCNN.

Abreha, H.G., Hayajneh, M. and Serhani, M.A., 2022. Federated learning in edge computing: a systematic survey. Sensors, 22(2), p.450.

Andrea, R., Ikhsan, N. and Sudirman, Z. (2022). Face Recognition Using Histogram of Oriented Gradients with TensorFlow in Surveillance Camera on Raspberry Pi. International Journal of Information Engineering and Electronic Business, 14(1), pp.46–52. doi:https://doi.org/10.5815/ijieeb.2022.01.05.

Arduino.cc. (2024). Available at: https://docs.arduino.cc/hardware/nano-33-ble-sense/#tech-specs [Accessed 3 Nov. 2024].

Arduino Help Center. (2024). Licensing for products based on Arduino. [online] Available at: https://support.arduino.cc/hc/en-us/articles/4415094490770-Licensing-for-products-based-on-Arduino [Accessed 10 Nov. 2024].

Beltrán, E.T.M., Gómez, Á.L.P., Feng, C., Sánchez, P.M.S., Bernal, S.L., Bovet, G., Pérez, M.G., Pérez, G.M. and Celdrán, A.H., 2024. Fedstellar: A platform for decentralized federated learning. Expert Systems with Applications, 242, p.122861.

Bonawitz, K., 2019. Towards federated learning at scale: System design. arXiv preprint arXiv:1902.01046.

Brahmbhatt, S., 2013. Practical OpenCV. Apress.

Carion, N. et al. (2020) 'End-to-End Object Detection with Transformers', CoRR, abs/2005.12872. Available at: http://arxiv.org/abs/2005.12872.

Ciaglia, F., Zuppichini, F.S., Guerrie, P., McQuade, M. and Solawetz, J.R., 100: A Rich, Multi-Domain Object Detection Benchmark. arXiv 2022. arXiv preprint arXiv:2211.13523.

Creative Commons (2016). Creative Commons — Attribution 4.0 International — CC BY 4.0. [online] Creativecommons.org. Available at: https://creativecommons.org/licenses/by/4.0/.

Ddrone, P. (2023) 'datasetDrone Dataset', Roboflow Universe [Preprint]. Roboflow. Available at: https://universe.roboflow.com/project-ddrone/datasetdrone-trv98.

Docker. (n.d.). Docker FAQs. [online] Available at: https://www.docker.com/pricing/faq/ [Accessed 6 Nov. 2024].

Docker Documentation. (2024). Docker Desktop license agreement. [online] Available at: https://docs.docker.com/subscription/desktop-license/ [Accessed 6 Nov. 2024].

Golzar, J., Noor, S. and Tajik, O., 2022. Convenience sampling. International Journal of Education & Language Studies, 1(2), pp.72-77.

Gou, J., Yu, B., Maybank, S.J. and Tao, D., 2021. Knowledge distillation: A survey. International Journal of Computer Vision, 129(6), pp.1789-1819.

Freund, R.J. and Wilson, W.J., 2003. Statistical methods. Elsevier.

Hartmann, J., Jueptner, E., Matalonga, S., Riordan, J. and White, S. (2022). Artificial Intelligence, Autonomous Drones and Legal Uncertainties. European Journal of Risk Regulation, 14(1), pp.1–18. doi:https://doi.org/10.1017/err.2022.15.

He, C., Annavaram, M. and Avestimehr, S., 2020. Group knowledge transfer: Federated learning of large cnns at the edge. Advances in Neural Information Processing Systems, 33, pp.14068-14080.

Jain, A., Bhattacharya, S., Masuda, M., Sharma, V. and Wang, Y. (2020). Efficient Execution of Quantized Deep Learning Models: A Compiler Approach. arXiv (Cornell University). doi:https://doi.org/10.48550/arxiv.2006.10226.

Kalra, S., Wen, J., Cresswell, J.C., Volkovs, M. and Tizhoosh, H.R., 2023. Decentralized federated learning through proxy model sharing. Nature communications, 14(1), p.2899.

Karras, A., Karras, C., Giotopoulos, K.C., Tsolis, D., Oikonomou, K. and Sioutas, S., 2022, September. Peer to peer federated learning: Towards decentralized machine learning on edge devices. In 2022 7th South-East Europe Design Automation, Computer Engineering, Computer Networks and Social Media Conference (SEEDA-CECNSM) (pp. 1-9). IEEE.

Knoche, H. and Eichelberger, H., 2018, March. Using the raspberry pi and docker for replicable performance experiments: Experience paper. In Proceedings of the 2018 ACM/SPEC International Conference on Performance Engineering (pp. 305-316).

Liu, S., Ha, D.S., Shen, F. and Yi, Y. (2021). Efficient neural networks for edge devices. Computers & Electrical Engineering, [online] 92, p.107121. doi:https://doi.org/10.1016/j.compeleceng.2021.107121.

Lu, W., Cheng, J., Li, X. and He, J. (2024). A Review of Solving Non-IID Data in Federated Learning: Current Status and Future Directions. Communications in computer and information science, pp.58–72. doi:https://doi.org/10.1007/978-981-97-1277-9_5.

Nguyen, Q., Pham, H.H., Wong, K.S., Le Nguyen, P., Nguyen, T.T. and Do, M.N., 2023. FedDCT: Federated Learning of Large Convolutional Neural Networks on Resource Constrained Devices using Divide and Collaborative Training. IEEE Transactions on Network and Service Management.

Northcutt, C.G., Athalye, A. and Mueller, J., 2021. Pervasive label errors in test sets destabilize machine learning benchmarks. arXiv preprint arXiv:2103.14749.

Openja, M., Majidi, F., Khomh, F., Chembakottu, B. and Li, H. (2022). Studying the Practices of Deploying Machine Learning Projects on Docker. The International Conference on Evaluation and Assessment in Software Engineering 2022. doi:https://doi.org/10.1145/3530019.3530039.

Ozkaya, O. and Yillikci, G., 2015. Arduino Computer Vision Programming. Packt Publishing.

Pandey, Jyoti and Abhijit R. Asati. (2023). Lightweight convolutional neural network architecture implementation using TensorFlow lite. International Journal of Information Technology 15 : 2489-2498. https://api.semanticscholar.org/CorpusID:259052888

Pau, D., Lattuada, M., Loro, F., De Vita, A. and Licciardo, G.D., 2021, January. Comparing industry frameworks with deeply quantized neural networks on microcontrollers. In 2021 IEEE International Conference on Consumer Electronics (ICCE) (pp. 1-6). IEEE.

Poda, X. and Qirici, O., 2018. Shape Detection and Classification Using OpenCV and Arduino Uno. RTA-CSIT, 2280, pp.128-136.

Radxa.com. (2024). Radxa ROCK 4C+. [online] Available at: https://radxa.com/products/rock4/4cp/ [Accessed 3 Nov. 2024].

Raspberry Pi. (n.d.). Licensing. [online] Available at: https://www.raspberrypi.com/licensing/ [Accessed 9 Nov. 2024].

Raspberry Pi (n.d.). Raspberry Pi 4 Model B specifications. [online] Raspberry Pi. Available at: https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/ [Accessed 3 Nov. 2024].

Rezende, J. da C.V., Silva, R.I. da and Souza, M.J.F. (2020). Gathering Big Data in Wireless Sensor Networks by Drone. Sensors, 20(23), p.6954. doi:https://doi.org/10.3390/s20236954.

Ridolfi, L., Naseh, D., Swapnil Sadashiv Shinde and Tarchi, D. (2023). Implementation and Evaluation of a Federated Learning Framework on Raspberry PI Platforms for IoT 6G Applications. Future Internet, 15(11), pp.358–358. doi:https://doi.org/10.3390/fi15110358.

Róbert Dávid, Duke, J., Jain, A., Vijay Janapa Reddi, Jeffries, N., Li, J., Kreeger, N., Nappier, I., Meghna Natraj, Shlomi Regev, Rhodes, R., Wang, T. and Warden, P. (2020). TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems. arXiv (Cornell University).

Roboflow.com. (2024). Terms. [online] Available at: https://roboflow.com/terms [Accessed 9 Nov. 2024].

roboflow (2022). GitHub - roboflow/roboflow-100-benchmark: Code for replicating Roboflow 100 benchmark results and programmatically downloading benchmark datasets. [online] GitHub. Available at: https://github.com/roboflow/roboflow-100-benchmark?tab=readme-ov-file [Accessed 1 Nov. 2024].

Rotem, N., Fix, J., Abdulrasool, S., Catron, G., Deng, S., Dzhabarov, R., Gibson, N., Hegeman, J., Lele, M., Levenstein, R., Montgomery, J., Maher, B., Nadathur, S., Olesen, J., Park, J., Rakhov, A., Smelyanskiy, M. and Wang, M. (2019). Glow: Graph Lowering Compiler Techniques for Neural Networks. [online] arXiv.org. doi:https://doi.org/10.48550/arXiv.1805.00907.

Shelke, M.S., Deshmukh, P.R. and Shandilya, V.K., 2017. A review on imbalanced data handling using undersampling and oversampling technique. Int. J. Recent Trends Eng. Res, 3(4), pp.444-449.

TensorFlow. (n.d.). TensorFlow Federated. [online] Available at: https://www.tensorflow.org/federated.

Ye, Y., Li, S., Liu, F., Tang, Y. and Hu, W., 2020. EdgeFed: Optimized federated learning based on edge computing. IEEE Access, 8, pp.209191-209198.

Yu, Y., Wang, C., Fu, Q., Kou, R., Huang, F., Yang, B., Yang, T. and Gao, M., 2023. Techniques and challenges of image segmentation: A review. Electronics, 12(5), p.1199.

Zhang, Jie & Qu, Zhihao & Chen, Chenxi & Haozhao, Wang & Zhan, Yufeng & Ye, Baoliu & Guo, Song. (2021). Edge Learning: The Enabling Technology for Distributed Big Data Analytics in the Edge. ACM Computing Surveys. 54. 1-36. 10.1145/3464419.

Zhang, P., Sun, H., Situ, J., Jiang, C. and Xie, D., 2021. Federated transfer learning for IIoT devices with low computing power based on blockchain and edge computing. Ieee Access, 9, pp.98630-98638.

APPENDIX

Terminology

DML - Distributed Machine Learning

EL - Edge Learning

FL - Federated Learning

CFL - Centralized Federated Learning

DFL - Decentralized Federated Learning

SBC - Single Board Computer

RASPI - Raspberry Pi

ROCKPI - Radxa Rock Pi

CV - Computer Vision

TF - Tensorflow

TF Lite - Tensorflow Lite (former name for LiteRT)

MCU - Microcontroller Unit

Hardware Technical Specification

	Raspberry Pi 4B	Rock Pi 4B+	Arduino Nano 33 BLE
SoC	Broadcom BCM2711	Rockchip RK3399-T	N/A, it's an MCU
CPU	Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.8GHz	Dual-Core ARM® Cortex-A72 @ 1.5GHz and Quad-Core ARM® Cortex-A53 @ 1.0GHz	nRF52840 64MHz
GPU	Broadcom BCM2711, supporting OpenGL ES 3.1, Vulkan 1.0	Arm Mali™ T860MP4 GPU, supporting OpenGL® ES 1.1 / 2.0 / 3.0 / 3.1 / 3.2, Vulkan® 1.0, OpenCL® 1.1 / 1.2, DirectX® 11.1	N/A
RAM	8GB LPDDR4-3200 SDRAM	LPDDR4, 4GB	256 KB SRAM, 1MB flash
Cooling	Active cooling and heat sinks		N/A

Table 2. Hardware Technical Specifications

Experimental Flowchart

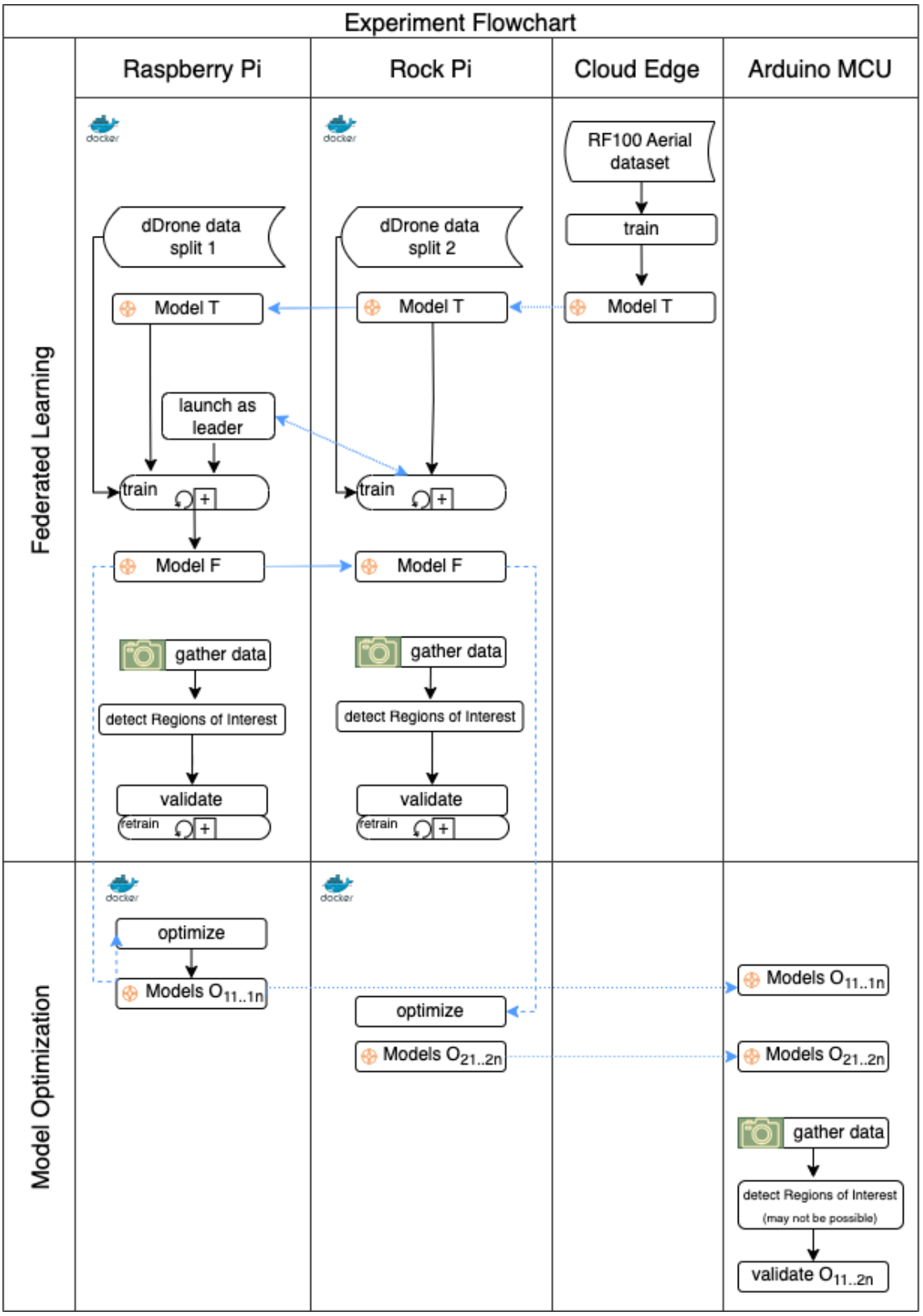

Figure 1. Experiments Flowchart